2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)

Integration of Jason Reinforcement Learning Agents into an Interactive Application

Costin Bădică, Alex Becheru
University of Craiova, Romania
Email: cbadica@software.ucv.ro, becheru@gmail.com

Samuel Felton
Institut National des Sciences Appliquées de Rennes, France
Email: samuel.felton@insa-rennes.fr

***Abstract*—The context of this paper is research on the application of agent-oriented programming to experiment with reinforcement learning algorithms. Traditionally, reinforcement learning fits well within the theory of agent systems. Nevertheless, most experimental approaches employ standard software engineering tools, rather than specific agent-oriented technologies developed within the agent community. The goal of our work is to stimulate synergies and development of agent-oriented programming technologies to better fit the context of agent systems research. In particular in this paper we focus on the development of an experimental interactive application that incorporates reinforcement learning agents created using the Jason agent-oriented programming language.**

I. Introduction

The results reported in this paper are part of a project aiming to stimulate the machine learning research in the context of the agent-oriented software engineering community that is interested in the development of software tools for agent-oriented programming. In particular the focus is on developing experimental reinforcement learning systems using the Belief-Desire-Intention (BDI) architecture and programming model of software agents.

Reinforcement Learning [1] - RL is a general learning paradigm that enables agents, single or in group, to learn how to adapt their performance in an uncertain and dynamic environment, in order to increase the value of their utility function. RL methods are basically classified into passive and active. Passive RL assumes that agents' action follows a fixed policy and their learning goal is to evaluate the utility function. Active RL assumes that agents aim to learn the optimal action policy to maximize their expected utility, while they are exploring an unknown environment.

The concept of "Agent-Oriented Programming" (AOP) was proposed more than 2 decades ago to describe "a new programming paradigm, one based on cognitive and societal view of computation" [2]. This proposal triggered research that led to the development of many AOP systems, more or less related to the initial AOP concept [3].

One of the earliest and most influential agent models is Belief-Desire-Intention - BDI, proposed 2 decades ago [4]. The key idea behind the BDI model is to endow agents with increased flexibility, understood as an appropriate mixture of reactive and proactive action, in order to meet practical requirements. Reactivity focuses on the agent responsiveness as capability of appropriately reacting to events, while proactivty is understood as agent's capability to exhibit a goal-directed behavior with minimal external intervention. This type of behavior is called "practical reasoning", i.e. reasoning directed towards action, and it is a typical trait of human behavior. It is interesting to note on one hand that the BDI model and the general RL methodology share similar goals using different approaches, while on the other hand there are very few works that looked into approaches of reconciliating these models.

The aim of this paper is to present our achievement regarding the development of an experimental system that integrates AOP and RL. This system is build around the Jason implementation based on Java platform of AgentSpeak(L) – a state-of-the-art AOP language using the BDI model [5]. This system integrates Jason agents for Temporal Difference Learning – TDL that we developed in our previous work [6], [7], as well updated Jason agents for Q-learning and SARSA [8].

Our work can be regarded as presenting a contribution towards narrowing the gap between RL and AOP, by considering TDL, Q-learning and SARSA as classic approaches of RL and AgentSpeak(L) / Jason as representative instance of AOP. In particular, our system shows how BDI agents programmed in Jason and endowed with RL skills can be integrated into larger non-agent applications. This can be used to illustrate the capabilities of AOP and RL in educational contexts. Moreover, it can serve as basis for developing more complex systems that incorporate Jason-based learning agents.

II. RL Background

Learning represents the ability of an agent to use the history of its action / perception experiences for improving its future performance. According to the RL paradigm, the agent is using the observed rewards (known also as reinforcements) which are part of its percepts, to learn an optimal policy for acting in an uncertain and dynamic environment [1].

RL abstracts the uncertainty and dynamics of the agent environment by adding nondeterminism to agent actions and Markov property to the environment model, i.e.:

- $E = [e, e', e'', \dots]$ is the set of environment states.
- $Ac = [a, a', a'', \dots]$ is the set of agent actions.
- The next environment state e' depends on the current state e and the agent action a, and the transition probability $p(e'|e, a)$ is known.
- Some of agent states are final or terminal, i.e no action can be further taken by the agent from those states. The

978-1-5386-2626-9/17/31.00 ©2017 IEEE
DOI 10.1109/SYNASC.2017.00065

agent evolution from the start state to a final state is called trial or episode, while an agent run comprises a sequence of trials.

For each environment state e the agent receives a percept as a pair $(e, R(e))$, where $R(e)$ is a real number that represents the agent's reward associated to state e. The agent behavior is defined by a policy $\pi : E \rightarrow Ac$, with $\pi(e)$ denoting the agent's action in state e defined by policy π.

As the agent environment is Markovian, multiple environment histories $H(e) = [e_0 = e, e_1, \dots]$ can be generated by an agent starting in a given initial state $e_0 = e$ and employing a given action policy π. Each history awards the agent with a given utility defined as $U_h([e_0, e_1, \dots]) = \sum_{i \geq 0} \gamma^i R(e_i)$. The parameter $\gamma \in [0, 1)$ represents the discount factor with the intuition that the reward r' perceived in the next state is equal with the reward r perceived as if the next state would have been the current state, adjusted with the multiplicative discount factor, i.e. $r' = \gamma r$. The utility obtained by the agent starting in environment state e and using policy π is defined as the expected value of the utility obtained for each possible environment history $U^\pi(e) = \mathbb{E}[U_h(H(e))]$.

Agent's goal in passive learning is to compute the values of function U^π for a given action policy π. For each policy π, function U^π satisfies Bellman system of equations (1).

$$U^\pi(e) = R(e) + \gamma \sum_{e' \in E} p(e'|e, \pi(e)) U^\pi(e') \text{ for } e \in E \quad (1)$$

It follows that values of U^π can be computed by solving system of equations (1). However, this is not always possible for realistic situations, as the agent might be in a "model-free" setting, i.e. it might not not posses the knowledge of the environment model. Sometimes the agent does not even know the set E of environment states, so it has to explore its unknown environment to discover at least (some of) the elements of E.

Using TDL, the agent observes each transition $e \rightarrow e'$ and adjusts the value of $U^\pi(e)$ to better approximate the solution of Bellman equations. The updated value is given by equation (2).

$$U^\pi(e) + \alpha(R(e) + \gamma U^\pi(e') - U^\pi(e)) \quad (2)$$

If the positive learning factor α is monotonically decreasing with the number $n(e)$ of times the environment is in state e, $\sum_{n \geq 0} \alpha(n) = \infty$ and $\sum_{n \geq 0} \alpha(n)^2 < \infty$ then TDL converges to the true value of $U^\pi(e)$ [8].

Agent's goal in active learning is to compute the optimal policy that maximizes the values of function U^π for each environment state. Let us denote the optimal policy with π^* and $U(e) = U^{\pi^*}(e)$. Function U satisfies Bellman system of equations $U(e) = R(e) + \gamma \max_{a \in Ac} \sum_{e' \in E} p(e'|e, \pi(e)) U(e')$. Now, assuming that $Q(e, a)$ is the utility of taking action a in environment state e, we obtain that $U(e) = \max_{a \in Ac} Q(e, a)$. Using the same idea as for the TDL agent in passive learning, we can incrementally adjust $Q(e, a)$ using the updated value given by equation (3).

$$Q(e, a) + \alpha(R(e) + \gamma \max_{a' \in Ac} Q(e', a') - Q(e, a)) \quad (3)$$

whenever action a is executed in state e leading to the new state e'. The corresponding algorithm is well-known in the literature as Q-learning. A closely related version of this update rule is State-Action-Reward-State-Action rule that uses the value defined by equation (4), thus obtaining the well-known SARSA algorithm.

$$Q(e, a) + \alpha(R(e) + \gamma Q(e', a') - Q(e, a)) \quad (4)$$

This update is operated at the end of a double transition $e \xrightarrow{a} e' \xrightarrow{a'} \dots$, i.e. a' is the actual action taken in state e'. Q-learning converges under the same assumptions as for the TDL approach of passive learning [9].

III. Related Works

An approach of developing passive TDL agents using the AOP paradigm based on BDI architecture was proposed in [6]. The main ideas behind this approach are as follows: i) separating the environment from the agent logic and their clean interaction based on a simple sense & act interface; ii) encoding the learning process using the BDI concepts – beliefs, goals, and plans. This approach has been extended in [7] to a multi-agent scenario in which various agents with different capabilities explore the same environment with the goal of estimating their utility.

Many software frameworks for the development of experiments using RL algorithms were already proposed in the Machine Learning community.

Quite similarly to our goals, researchers were interested to develop better and simpler abstractions of RL. A language independent software framework for reinforcement learning experiments entitled RL-Glue was proposed in [10]. The main idea behind RL-Glue was to develop an abstract interface and protocol, as well as a mediation software that allows the clean and standardized interaction between agents and environments in RL experiments.

RLPy is an object-oriented value-function-based RL framework for education and research written in Python and focused on linear function approximation methods and discrete agent actions [11]. The main motivations that geared the development of RLPy were also unification and reusability of RL software frameworks. The main advantages of RLPy are the easy accessibility for students and novices in RL, as well as the efficiency and efficacy for RL researchers. RLPy shares similar goals with our work, with the difference that our proposal aims also to bridge the gap between AOP and RL communities. Moreover, RLPy is focused on value-function-based RL based on linear function approximation that is not currently supported by our system.

RL4J is a deep RL framework that mainly targets environments characterized by highly-dimensional state representations, as typically encountered in video games [12]. RL4J is integrated with the Deep Learning for Java library – DL4J, to be able to address the problems associated to high dimensionality of pixel representations of video game frames.

The Brown-UMBC Reinforcement Learning and Planning (BURLAP) is a Java library for the use and development of

single or multi-agent planning and learning algorithms, as well as their domains [13]. The motivation was the need to develop RL algorithms on top of a more expressive environment representation incarnated into the object-oriented MDP formalism. Currently BURLAP allows finite, infinite, continuous, and relational state spaces in single and multi-agent settings.

The BDI approach in general and AgentSpeak(L) in particular were important sources of inspiration for the development of new AOP frameworks and languages.

Reflecting on the cognitive gaps between AOP and standard OOP reported by software engineers, the new programming language ASTRA reconciliating the OOP and AOP paradigms was proposed in [14]. Taking into account that the main source of inspiration for ASTRA was AgentSpeak(L), we have strong reasons to believe that our approach is also supported by ASTRA. Moreover, according to the recent review [3], a multitude of agent-oriented programming languages and platforms have been proposed by the agent research community, with many of them following the BDI paradigm (like Jadex, Jack, and Goal). Therefore, at least in principle our approach could be adapted for these language and platforms.

A new Python framework entitled PROFETA, standing for Python RObotic Framework for dEsigning sTrAtegies, for programming autonomous robots using a declarative approach was proposed in [15]. The main contribution of this work is to provide a methodological and technical solution for extending Python with AgentSpeak(L) constructs. This suggests that, at least in principle, our approach can be also implemented using PROFETA.

A new hybrid architecture of agents acting in an uncertain environment that combines the BDI framework with partially observable Markov decision problems – POMDP was proposed in [16]. The BDI-POMDP architecture aims to improve the traditional POMDP approach by leveraging on the BDI concepts. This is different from our work where we add RL facilities on top of the standard BDI concepts, rather than mixing the two approaches into a new cognitive architecture.

IV. System Architecture

The system presented here was created starting from our previous work that introduced an approach of developing BDI agents for TDL using Jason [6], [7]. The system was designed to serve the purpose of assisting researchers in the task of experimenting with BDI learning agents. It includes a framework for expressing RL algorithms under the umbrella of BDI paradigm and AOP technologies, thus enabling synergies between learning and agent communities.

The general system setting consists in RL agents situated in a two dimensional grid environment [8]. Agents can move up, down, left and right. However, the result of the agents' action is stochastic, i.e. following their action, agents can move successfully according to the direction of their action, or erroneously with given probabilities, i.e. by slightly deviating to left or right from the direction of their action. This behavior is aimed to capture the agent's uncertainty about the environment.

Our initial proposal introduced in [6], [7] was extended in several directions, including: i) upgrade of the passive TDL agent to active RL agents based of Q-learning and SARSA approaches; ii) add support for interactive edit of the grid-based prototype agent environment; iii) add support for visualization of agents' performance.

The system architecture is presented in Figure 1. It is conceptually partitioned into the following layers:

i) *Agent environment*. It is responsible with interfacing the agents' with the outside world.
ii) *Agent code*. It contains the specification of the RL algorithms using the BDI and AOP paradigms.
iii) *Environment GUI*. It allows the interactive creation and monitoring of agent-based RL experiments.
iv) *Agent – GUI integration code*. It allows exchange of information between the agent Jason code and the GUI Java code.
v) *Visualization layer*. It provides features for visualizing the agents' performance during a given experiment.

In the next section of this paper we introduce the components of the system and we outline their functionalities[1].

V. System Components

A. Agent environment

This component is responsible with the management of the agents' environment. Basically this includes the functions for storing and updating the environment state in response to agent actions. The code of this component was developed by refactoring our initial proposal presented in [6]. The code was dispersed in a number of classes to allow: interfacing with the Jason interpreter (class *MDPEnv*), storing and updating of environment state (class *MDPModel*), and visualizing inside the system GUI (class *MDPView*).

B. Environment GUI

This interface is responsible with the management of the agent environment. Firstly, this GUI allows to create and edit a two dimensional environment grid composed of cells of several types: free cell (agents can move in an out of this cell), obstacles (agents cannot step in this cell), positive terminal state (to denote an agent successful or goal state), and negative terminal state (to denote an agent failure or unsuccessful state). Additionally, the GUI allows to input the value of the reward associated to each cell. The environment can be saved and latter reloaded using the Java object serialization mechanism.

Secondly the GUI is responsible with the creation and visualization of agent policies. For the passive TDL agent that is characterized by a static (i.e. fixed) policy, this GUI allows the creation of the policy by editing the agent move associated to each cell of the environment grid. For the active Q-learning and SARSA agents that are characterized by dynamic (i.e. variable) policies, the requirement is to allow visualization of the change of the agent policy during the exploration process.

[1]The code is available on GitHub at the following URL: https://github.com/IntelligentDistributedSystems/SamuelFelton.

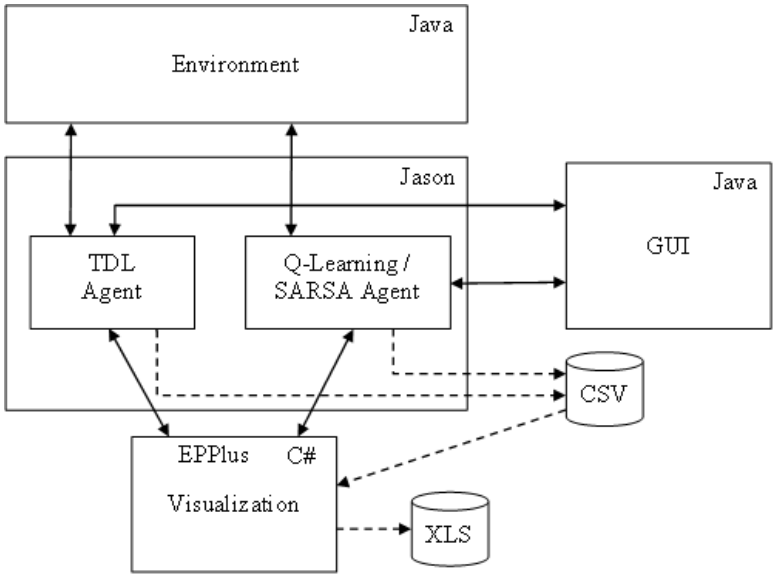

Fig. 1. System architecture

Finally, this GUI allows to save at the end of the exploration process, the evolution of the agent state-action valuations and agent moves, for the latter analysis and visualization.

Figure 2 presents the *Environment GUI* in action for the passive TDL agent. This figure displays an environment grid of size 7×8. The grid is bordered by default with obstacles. Other 4 obstacles are set inside the grid. The rest of 26 the grid cells are free cells. Assuming that the bottom leftmost cell is on row 0 and column 0, among the cells from Figure 2 there is one goal cell (marked with green – cell at row 3 and column 4) and one failure cell (marked with red – cell at row 4 and column 4). For 6 free cells the policy has been already input – see the little arrow indicating the agent move associated to that cell. The cell at row 2 and column 5 marked in blue is currently selected by the user, allowing him or her to input the associated policy move and reward value.

For all the cells, the field marked with 'T' displays the number of trials recorded by the cell. This number is initially set to 0. The right hand panel of the GUI contains a set of controls that allow the user to input the type, policy move, and reward of the currently selected grid cell. The 'Run' button can be used to start the passive TDL agent exploration process. The upper left menu contains a set of commands useful for initializing and (re-)sizing the environment grid, as well as for saving and restoring a given environment grid.

The *Environment GUI* for the active agents has been slightly updated. Firstly, a new command for saving the Q-values at the end of an experiment, has been added to the upper left menu. Secondly, each cell now displays the current value of the policy move as well as the associated Q-value. Finally, right-clicking a cell displays a popup window containing a summary information for that cell: number of trials of each possible move and its associated Q-value. An example is shown in Figure 3.

C. Agent code

The Jason code of the various learning agents follows the general model of an agent exploring an unknown Markovian environment that was introduced in [6]. Firstly, the passive TDL agent has been slightly updated and integrated into our system. Then, the Jason code has been updated to support active RL agents based on Q-learning and SARSA, while the resulting agents have been also integrated into our system.

The major contribution, as compared to our previous work [6], was the development of the new Q-learning and SARSA agents. These agents share the same structure that has been now factored out into a separate Jason code defining the basics of an active RL agent and the part that is specific to the active RL algorithm. The common part is imported by both Q-learning and SARSA agents using Jason's $\{\texttt{include}(\texttt{"learningAgent.asl"})\}$ feature.

Code listings from Table I present the specific Jason code of active RL agents to perform the update of Q-values using equations (3) for the Q-learning agent (Subtable Ia) and (4) for the SARSA agent (Subtable Ib).

Note in Table I that the Q-learning agent is using a fixed learning rate $\alpha = 0.9$, while the SARSA agent is using a variable learning rate $\alpha(n) = \frac{60}{59+n}$, where n (captured by variable $M1$) represents the number of A actions executed in the last visited state St.

Our active RL agents are able to balance their exploration and exploitation functions using the ϵ-greedy action selection strategy [8].

Currently, the system is able to support multiple Q-learning and SARSA agents, as shown in Listing 1. This listing presents the description of a Jason multi-agent system comprising 5 Q-

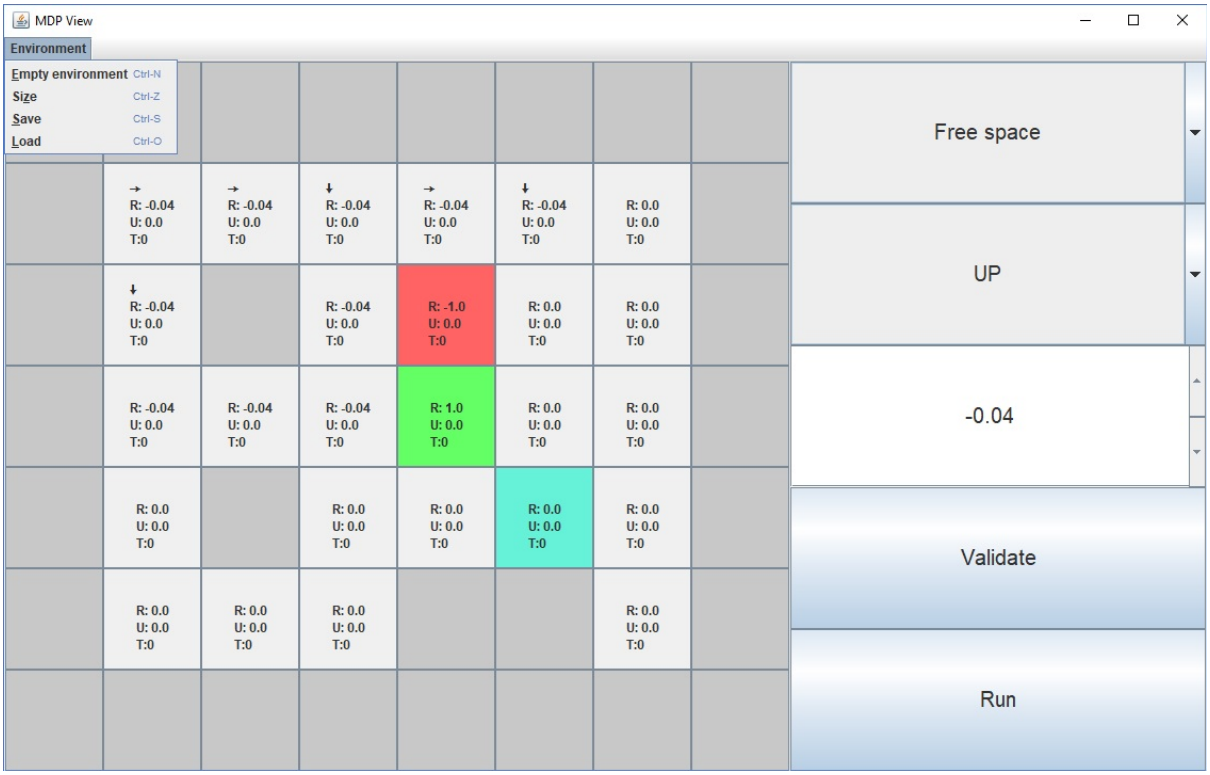

Fig. 2. Environment GUI

TABLE I
JASON CODE FOR Q-VALUES UPDATE IN ACTIVE RL AGENTS

```
+!update_qvalue(St1,_,St,R,Q_St,A,M1) :
   non_terminal_state(St) <-
   .findall(Q,qvalue(St1,A1,Q,_),Qs);
   ?maxim_list(Qs,Q_St1);
   ?gamma(Discount);
   Q1 = Q_St + 0.9*(R+Discount*Q_St1-Q_St);
   -qvalue(St,A,_,_);
   +qvalue(St,A,Q1,M1).
+!update_qvalue(_,R1,St,_,_,null,M1) :
   terminal_state(St) <-
   -qvalue(St,null,_,_);
   +qvalue(St,null,R1,M1).
```

a. Q-learning agent

```
+!update_qvalue(St1,A1,_,St,R,Q_St,A,M1) :
   non_terminal_state(St) <-
   ?gamma(Discount);
   LearningRate = 60.0/(M1+59.0);
   ?qvalue(St1,A1,Q_St1,N2);
   Q1 = Q_St + LearningRate *(R+Discount*Q_St1-Q_St);
   -qvalue(St,A,_,_);
   +qvalue(St,A,Q1,M1).
+!update_qvalue(_,_,_,St,R,_,null,M1) :
   terminal_state(St) <-
   -qvalue(St,null,_,_);
   +qvalue(St,null,R,M1).
```

b. SARSA agent

learning agents and 5 SARSA agents interacting with an agent environment specified by the Java class `mdp.MDPEnv` and using a centralised infrastructure (i.e. all the agents are created on the same machine).

```
MAS q_Learning_Sarsa {
   infrastructure: Centralised
   environment: mdp.MDPEnv
   agents:
      qAgent #5;
      sarsaAgent #5;
   aslSourcePath:
      "src/asl";
}
```

Listing 1. Specification of multiple learning agents

D. Agent – GUI integration code

The major aspect of the integration was the interfacing of Jason code with the GUI code. Firstly, the static policy that is input via the GUI must be passed to the agent. Initially, this policy was hardcoded into Jason code. However, this ad-hoc solution was not satisfactory for the interactive system. Moreover, the agent start is now controlled by the GUI. So, in this situation agents are programmed such that they do not start immediately. The start of their execution is explicitly triggered by the user via the GUI. The user action generates a percept `must_run` that is handled by the agent. Similarly, the policy input by the user via the GUI triggers a percept `policy(State, Action)` that is handled by the agent. The corresponding Jason code is presented in Listing 2. Note

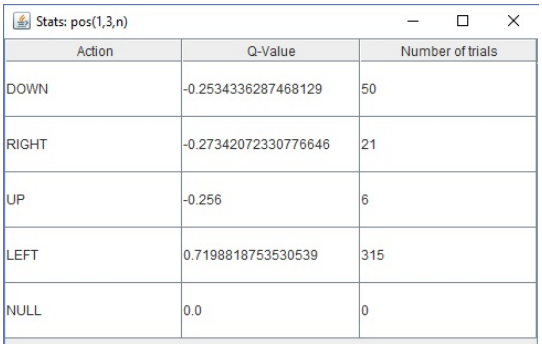

Fig. 3. Inspecting the Q-values of a given cell

that start of the agent execution is controlled by explicitly invoking the !start achievement goal, differently from [6], where !start was implicitly invoked immediately after the agent initialization.

```
+policy(s(R,C),D)[source(percept)] <-
   -policy(s(R,C),_)[source(percept),source(self)];
   +policy(s(R,C),D).
+must_run[source(percept)] <-
   !start;
   -must_run[source(percept)].
```

Listing 2. Policy receive and agent start

A snapshot of the Java code for creating and adding the corresponding percepts to the agents' environment is presented in Listing 3. For example, inside method addPolicy, the policy item corresponding to a given grid cell is packed into a Jason term representing a policy percept comprising an environment state and agent action pair, and then it is "added" to the set of percepts of the agent. The set of percepts representing all policy items that define the agent action for each environment state is added to the agents' environment using method sendPolicies.

Secondly, values of variables that are continuously changing during the exploration process in result to agent actions, including utilities (for the passive agent) and state-action values (for active agents), number of trials (for all type of agents), as well as the agent policy (for dynamic agents) must be conveniently visualized into the GUI. This requires the passing of information in opposite direction, from the agent to the GUI. This task is achieved by defining two agent actions send_policies and send_utilities. A snapshot of the corresponding code in Java and Jason is presented in Listing 4. The Jason agent action send_utilities is handled by the getUtilities Java method that iterates through the list of utilities, extracts the corresponding values and then packs them into corresponding GridState objects.

E. Visualization

The visualization function aims to provide the user with some useful insight regarding the evolution of the learning process. The visualization is achieved in two steps: data export and graphics generation.

The first step is responsible with the data export, with the aim of saving the successive Q-values, as well as the number of action tried by the agent for each environment state, in a simple format to facilitate further processing for the visualization purpose. We have chosen the comma-separated values (CSV) text format, which is simple to generate and process, as well as easy to use by advanced data processing and visualization tools, like Microsoft Excel. The export function works as follows: firstly, an object of class AgentData is created for each agent, and then, at the end of the learning process, the data is exported to a text file in CSV format, for each agent in the system.

The second step is responsible with the generation of graphics. Based on the compatibility with the CSV format, as well as on its flexibility, we have chosen the Excel spreadsheet format as target for the graphics output. This task was achieved by developing a separate tool using the C# programming language and the EPPlus open-source package [2] for programmatically manipulating Excel files.

The visualization function generates three types of graphics, for each agent in the system:

i) A representation of the evolution of the Q-values for each state..
ii) A representation of the evolution of the number of trials for each state.
iii) A pie chart illustrating the final repartition of the number of trials for each state.

Figure 4 presents a comparison of the evolution of Q-values for each successive agent move taken in state $s(1,4)$. It can be easily noticed that the total number of agent moves was 23000, and that the most preferred move was by far $left$.

Figure 5 presents a comparison of the evolution of the number of trials for each successive agent move taken in state $s(1,4)$. Note that the number of trials for moves up, $right$ and $down$ has an upper bound, while the number of trials for move $left$ grows continuously. This fact shows that starting with some iteration the preferred move is constantly $left$.

Figure 6 presents the final repartition of the number of trials corresponding to each agent move for state $s(1,4)$.

VI. Conclusion

This paper introduced an experimental interactive software tool to support experiments with RL agents under the umbrella of AOP based on Jason language. The system incorporates Jason agents that use basic algorithms for passive TDL, as well as active Q-learning and SARSA. The system is modular and easy extensible with a clean separation of the agent environment, agent logic based on BDI approach, GUI and visualization parts. Our initial experiments support the usability of the AOP paradigm and BDI approach for the development of RL agents.

[2]http://epplus.codeplex.com/

```
public void askAgentToRun() {
   updatePercepts();
   sendPolicies();
   addPercept(agentName, ASSyntax.createLiteral("must_run"));
}

public void sendPolicies() {
   for (final Map.Entry<Coordinates, GridState> entry : model.getStates().entrySet()) {
      addPolicy(agentName, entry.getValue().getAction(), entry.getKey());
   }
}

public void addPolicy(String agName, AgAction a, Coordinates c) {
   final String action = getActionTextForAgent(a);
   final Literal s = ASSyntax.createLiteral("s", ASSyntax.createNumber(c.y), ASSyntax.createNumber(c.x));
   final Literal l = ASSyntax.createLiteral("policy", s, ASSyntax.createAtom(action));
   addPercept(agName, l);
}
```

Listing 3. Java code for creating percepts that trigger agent start

```
public boolean executeAction(String ag, Structure action) {
   ...
   if (action.getFunctor().equals("send_utilities")) {
      getUtilities(ag, action);
   }
   ...
   return true;
}

private boolean getUtilities(String ag, Structure action) {
   ...
   final List<Term> listUtilities = ((ListTermImpl) (action.getTerm(0))).getAsList();
   for (final Term utility : listUtilities) {
      final Literal literalUtility = (Literal) utility;
      final Literal posLiteral = (Literal) literalUtility.getTerm(0);
      final NumberTerm ntRow = (NumberTerm) posLiteral.getTerm(1);
      final NumberTerm ntCol = (NumberTerm) posLiteral.getTerm(0);
      final NumberTerm u = (NumberTerm) literalUtility.getTerm(1);
      final NumberTerm t = (NumberTerm) literalUtility.getTerm(2);

      final Coordinates posState = new Coordinates(Integer.parseInt(ntRow.toString()),
      Integer.parseInt(ntCol.toString()));
      final GridState gs = model.getState(posState);
      gs.setTrialNumber(Integer.parseInt(t.toString()));
      gs.setUtility(Double.parseDouble(u.toString()));
   }
   return true;
}

+!send_utilities <-
   .findall(u(St,U,N),utility(St,U,N),UL);
   send_utilities(UL).
```

Listing 4. Actions for sending policies and utilities

ACKNOWLEDGMENT

The results reported in this paper were obtained during the Erasmus training stage that was carried out by Samuel Felton (while he was student at the University of Limoges, France) in the Intelligent Distributed Systems research group – IDS[3] of the University of Craiova, Romania during the second semester of the academic year 2015–2016.

REFERENCES

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, ser. Adaptive Computation and Machine Learning. Cambridge, MA,: A Bradford Book. The MIT Press, 1998.

[2] Y. Shoham, "Agent-oriented programming," *Artificial Intelligence*, vol. 60, no. 1, pp. 51–92, Mar. 1993. [Online]. Available: https://doi.org/10.1016/0004-3702(93)90034-9

[3] C. Bădică, Z. Budimac, H. Burkhard, and M. Ivanović, "Software agents: Languages, tools, platforms," *Computer Science and Information Systems*, vol. 8, no. 2, pp. 255–298, 2011. [Online]. Available: http://dx.doi.org/10.2298/CSIS110214013B

[4] A. S. Rao, "Agentspeak(l): Bdi agents speak out in a logical computable language," in *Agents Breaking Away: 7th European Workshop on Modelling Autonomous Agents in a Multi-Agent World, MAAMAW '96 Eindhoven, The Netherlands, January 22–25, 1996 Proceedings*, ser. Lecture Notes in Computer Science, W. Van de Velde and J. W. Perram, Eds. Springer Berlin Heidelberg, 1996, vol. 1038, pp. 42–55. [Online]. Available: http://dx.doi.org/10.1007/BFb0031845

[5] R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*, ser. Wiley Series in Agent Technology. John Wiley & Sons, 2007.

[6] A. Bădică, C. Bădică, M. Ivanović, and D. Mitrović, "An approach of temporal difference learning using agent-oriented programming," in *20th Int. Conf. Control Systems and Computer Science – CSCS'2015*, 2015, pp. 735–742. [Online]. Available: http://dx.doi.org/10.1109/CSCS.2015.71

[7] A. Bădică, C. Bădică, M. Ganzha, M. Ivanović, and M. Paprzycki,

[3] http://ids.software.ucv.ro

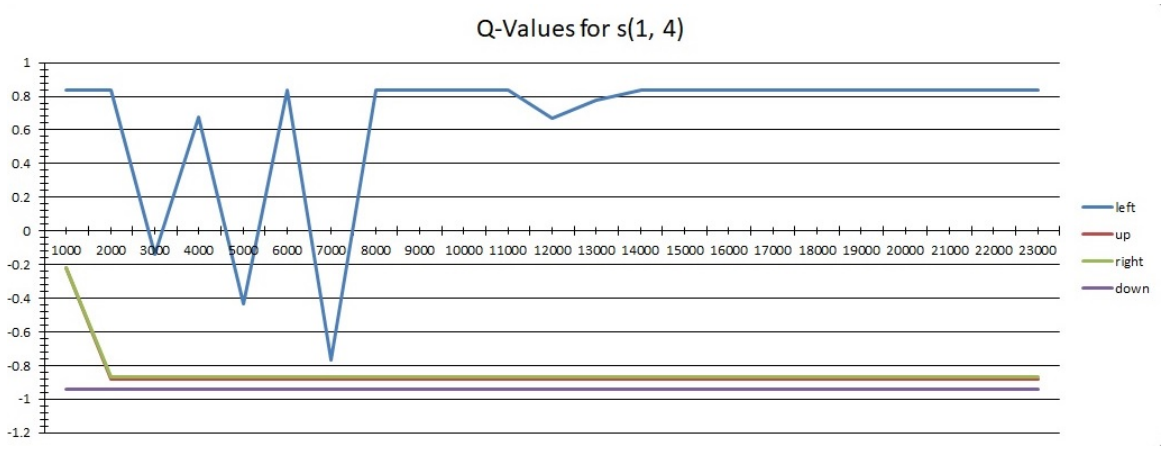

Fig. 4. Comparing the evolution of Q-values for state $s(1, 4)$

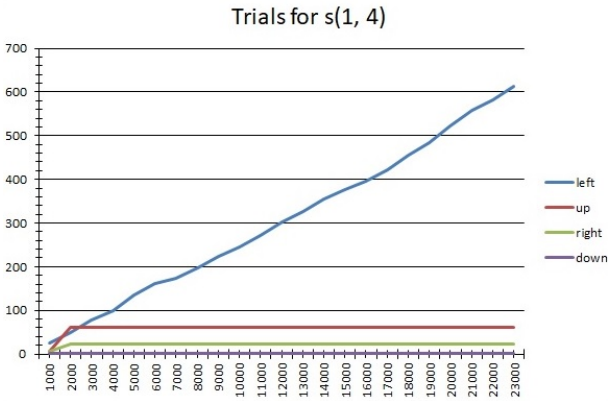

Fig. 5. Comparing the evolution of number of trials for state $s(1, 4)$

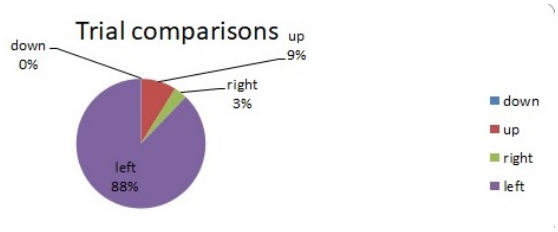

Fig. 6. Final repartition of number of trials for state $s(1, 4)$

"Experiments with multiple BDI agents with dynamic learning capabilities," in *Highlights of Practical Applications of Scalable Multi-Agent Systems. The PAAMS Collection - International Workshops of PAAMS 2016*, ser. Communications in Computer and Information Science, J. Bajo, M. J. Escalona, S. Giroux, P. Hoffa-Dabrowska, V. Julián, P. Novais, N. S. Pi, R. Unland, and R. A. Silveira, Eds. Springer, 2016, vol. 616, pp. 274–286. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-39387-2_23

[8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., ser. Prentice Hall Series in Artificial Intelligence. Prentice Hall, 2010.

[9] F. S. Melo, "Convergence of q-learning: a simple proof," 2007. [Online]. Available: http://users.isr.ist.utl.pt/~mtjspaan/readingGroup/ProofQlearning.pdf

[10] B. Tanner and A. White, "RL-Glue: Language-independent software for reinforcement-learning experiments," *Journal of Machine Learning Research*, vol. 10, pp. 2133–2136, September 2009. [Online]. Available: http://www.jmlr.org/papers/volume10/tanner09a/tanner09a.pdf

[11] A. Geramifard, C. Dann, R. H. Klein, W. Dabney, and J. P. How, "Rlpy: A value-function-based reinforcement learning framework for education and research," *Journal of Machine Learning Research*, vol. 16, pp. 1573–1578, 2015. [Online]. Available: http://jmlr.org/papers/v16/geramifard15a.html

[12] R. Fiszel, "Reinforcement learning and dqn, learning to play from pixels," 2016. [Online]. Available: https://rubenfiszel.github.io/posts/rl4j/2016-08-24-Reinforcement-Learning-and-DQN.html

[13] J. MacGlasha, "Brown-umbc reinforcement learning and planning (burlap)." [Online]. Available: http://burlap.cs.brown.edu/

[14] R. W. Collier, S. Russell, and D. Lillis, "Reflecting on programming with agentspeak(l)," in *Proceedings of 18th International Conference on Principles and Practice of Multi-Agent Systems (PRIMA2015)*, ser. Lecture Notes in Computer Science. Springer, 2015, vol. 9387, pp. 351–366. [Online]. Available: https://doi.org/10.1007/978-3-319-25524-8_22

[15] L. Fichera, F. Messina, G. Pappalardo, and C. Santoro, "A python framework for programming autonomous robots using a declarative approach," *Science of Computer Programming*, vol. 139, pp. 36–55, 2017. [Online]. Available: https://doi.org/10.1016/j.scico.2017.01.003

[16] G. Rens and D. Moodley, "A hybrid pomdp-bdi agent architecture with online stochastic planning and plan caching," *Cognitive Systems Research*, vol. 43, pp. 1–20, 2017. [Online]. Available: https://doi.org/10.1016/j.cogsys.2016.12.002